12 OPERATOR OVERLOADING • THEORY

01 operator overloading fundamentals and rules

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

01 — Operator Overloading: Fundamentals, Syntax, and Strict Rules

1. Definition & Intuition

Operator Overloading is a compile-time polymorphism feature in C++ that allows programmers to define custom meanings for existing C++ operators (`+` , `-` , `*` , `==` , `++` , `<<` , etc.) when applied to user-defined types (`class` or `struct`).

Why Operator Overloading is Needed

In Mathematics and Computing, we work with mathematical entities such as Complex Numbers, Matrices, Vectors, Polynomials, and Rational Fractions.

- Without operator overloading, adding two complex numbers requires verbose function calls: `c3 = c1.add(c2);`
- With operator overloading, we write natural mathematical notation: `c3 = c1 + c2;`

2. Syntax of an Overloaded Operator Function

CPP

```
return_type operator@(parameter_list) {
    // implementation
}
```

Here `@` represents any overloadable C++ operator symbol (such as `+` , `-` , `*` , `++` , `==`).

3. Operators That CANNOT Be Overloaded (High-Frequency Exam Question)

The following C++ operators **cannot** be overloaded under any circumstances (`[over.oper]`):

1. **Scope Resolution Operator (`::`)** — Operates on compile-time names/scopes, not runtime values.
2. **Member Access / Dot Operator (`.`)** — Preserves unambiguous access to actual class members.
3. **Pointer-to-Member Operator (`.*`)** — Preserves built-in pointer-to-member resolution.
4. **Ternary Conditional Operator (`?:`)** — Has short-circuit evaluation semantics for its second and third operands that normal function argument evaluation cannot replicate.

5. **Object Size Operator (`sizeof`)** — Evaluated at compile time by the compiler.
6. **Type Identification Operator (`typeid`)** — Built-in RTTI mechanism.
7. **Alignment & Exception Operators (`alignof`, `noexcept`)** — Compile-time type/expression queries.

---

4. Operators That MUST Be Non-Static Member Functions

The C++ standard mandates that these four operators can **only** be overloaded as **non-static member functions** (they **cannot** be overloaded as `friend` or free non-member functions):

1. Assignment operator: `=`
2. Array subscript operator: `[]`
3. Function call operator: `()`
4. Member access arrow operator: `→`

---

5. Fundamental Rules & Restrictions of Operator Overloading

1. **No New Operator Symbols**: You cannot invent new tokens like `**` for exponentiation or `<>` for not-equal.
2. **Precedence and Associativity Never Change**: Overloading `*` and `+` for matrices will still evaluate `A + B * C` as `A + (B * C)` with the exact same built-in precedence and associativity.
3. **Arity (Number of Operands) Never Changes**: A unary operator cannot take two operands, and a binary operator cannot take three operands.
4. **No Default Arguments** (except for `operator()`): Overloaded operator functions cannot have default arguments (`operator+(int x = 0)` is a compilation error).
5. **At Least One User-Defined Operand**: You cannot redefine how `+` works on two built-in `int` values (`int operator+(int, int)` is illegal).